

PropertyPilot — Maintenance and Setup Guide

For: Technical operators deploying, running, or maintaining the PropertyPilot instance. **Last updated:** 2026-06-02

This document covers everything needed to take a fresh clone of the repository to a running local instance, run the test suite, build a production bundle, and deploy.

1. Prerequisites

Requirement	Version	Why
Node.js	20 LTS or newer	Backend runtime, build tooling, npm workspaces
npm	10 or newer	Ships with Node 20+
Docker Desktop	Latest stable	Runs the local PostgreSQL 16 container
Git	Any recent	Cloning the repo
PowerShell	Windows PowerShell 5.1 or newer	The command examples below use PowerShell syntax; bash works on macOS/Linux with equivalent commands

Verify everything is installed:

```
node --version
npm --version
docker --version
docker compose version
git --version
```

If `docker compose` is missing but `docker` works, reinstall Docker Desktop — older standalone installs sometimes omit Compose v2.

2. Repository Layout

```
/
├─ backend/                # Express + Prisma API
│  └─ prisma/
│     ├── schema.prisma    # Domain models, indexes, enums
│     ├── migrations/      # Versioned migrations
│     └─ seed.ts           # Dev-only seed script
│  └─ src/
│     ├── domain/          # Domain classes + mappers
│     ├── repositories/    # Owner-scoped Prisma access
│     ├── controllers/     # HTTP handlers
│     ├── routes/          # Express route wiring
│     ├── middleware/      # Auth, error handling
│     ├── schemas/         # Zod request schemas
│     └─ reports/          # Rent Roll, P&L, Occupancy, Aging
```

```

| | | └─ forecast/          # CashFlowForecaster
| | |   └─ app.ts           # Express bootstrap
| | └─ package.json
| |   └─ tsconfig*.json
└─ frontend/               # Vite + React app
  └─ src/                  # Pages, components, lib, schemas
    └─ package.json
    └─ vite.config.ts
└─ docs/                   # Design doc, guides, test plan/results
└─ docker-compose.yml      # Local Postgres container
└─ .env.example            # Template for the real .env
└─ package.json            # Workspace root
└─ README.md

```

The root `package.json` defines a two-workspace setup (`backend` , `frontend`). All commands below operate on those workspaces.

3. Local Setup with Docker Postgres

3.1 Clone

```

git clone https://gitlab.com/wgu-gitlab-environment/student-repos/mriehm1/d424-software-
engineering-capstone.git propertypilot
cd propertypilot

```

3.2 Create the local `.env`

Copy-Item `.env.example .env`

The default values in `.env.example` line up with `docker-compose.yml` , so the file is usable as-is for local development. The four variables are documented in section [4. Environment Variables](#) below.

3.3 Start PostgreSQL

```

docker compose up -d

```

This starts a single container named `propertypilot-postgres` running `postgres:16-alpine` , exposed on `localhost:5432` . The credentials are `propertypilot / propertypilot` and the database name is `propertypilot` . Data persists in a named Docker volume (`propertypilot_pgdata`) that survives container restarts.

Verify the container is healthy:

```

docker compose ps

```

You should see the container in the `running (healthy)` state. The healthcheck runs `pg_isready` every 5 seconds.

3.4 Install dependencies

From the repository root:

```
npm install
```

npm workspaces install root, backend, and frontend dependencies in one pass.

3.5 Run migrations

```
npm run prisma:migrate -w backend
```

This applies every migration under `backend/prisma/migrations/` to the local Postgres. It also runs `prisma generate` so the client typings match the schema.

If you want the dev seed data (one demo landlord, two properties, sample leases and transactions):

```
npm run db:seed -w backend
```

The seed creates a user with email `dev@propertypilot.local` and password `dev1234`. Use it to log in immediately without registering.

3.6 Start the dev servers

```
npm run dev
```

From the repository root, the concurrent runner launches both workspaces:

- Backend at <http://localhost:4000>
- Frontend at <http://localhost:5173>

The Vite dev server proxies `/api` to the backend, so the browser only ever needs to know the frontend URL. CORS on the backend is restricted to `FRONTEND_URL` (defaults to `http://localhost:5173`).

To stop the dev servers, press `Ctrl + C` in the terminal.

4. Environment Variables

All variables live in a single `.env` at the repository root. Backend reads them via `dotenv` at startup; Prisma reads `DATABASE_URL` directly.

Variable	Required	Example	Note
DATABASE_URL	Yes	postgresql://propertypilot:propertypilot@localhost:5432/propertypilot?schema=public	Prisma connect string. It includes Docker's matches

			example of the bc
JWT_SECRET	Yes	<long-random-string>	Signs and verifies JWTs. Must be at least 16 characters. The backend refuses to start. Remember to generate this before a non-local deployment.
PORT	No	4000	Backend listens on this port. Defaults to 4000.
FRONTEND_URL	No	http://localhost:5173	Origin allowed by the backend CORS middleware. Set this to the URL of the production frontend when deploying.

Generating a secure JWT secret

```
[Convert]::ToBase64String([System.Security.Cryptography.RandomNumberGenerator]::GetBytes(48))
```

Paste the output into `JWT_SECRET`. **Never commit a real secret.** The `.gitignore` excludes `.env`; `.env.example` is the only file in source control.

5. Running Migrations

5.1 Apply migrations

```
npm run prisma:migrate -w backend
```

In dev mode this runs `prisma migrate dev`, which:

1. Applies any pending migration.
2. Regenerates the Prisma client.

3. Prompts you for a migration name if the schema has changed but no migration exists for the change.

5.2 Create a new migration

Edit `backend/prisma/schema.prisma`, then run:

```
npm run prisma:migrate -w backend
```

Prisma will detect the diff, ask for a name, and write a new folder under `backend/prisma/migrations/`. Commit the folder along with the schema change.

5.3 Inspect the database

```
npm run prisma:studio -w backend
```

Opens Prisma Studio at `http://localhost:5555` for ad-hoc browsing.

5.4 Reset the database (destructive)

If you need a clean slate locally:

```
docker compose down -v
docker compose up -d
npm run prisma:migrate -w backend
npm run db:seed -w backend
```

The `-v` flag deletes the named Docker volume so the new container starts empty. **Only run this against the local database.**

6. Starting the Dev Servers

6.1 Both at once

```
npm run dev
```

Uses `concurrently` to run `npm:dev -w backend` and `npm:dev -w frontend` in parallel. Output is prefixed `backend |` and `frontend |`.

6.2 Individually

```
# Backend only (auto-restarts on save via ts-node-dev)
npm run dev -w backend

# Frontend only
npm run dev -w frontend
```

6.3 Health check

```
Invoke-RestMethod http://localhost:4000/api/health
```

Should return a small JSON status payload. If it fails, the backend has not finished starting yet or `JWT_SECRET` is not set.

7. Running Tests

The test suite is Vitest only; no Jest, no Mocha. As of this release the suite is 209 tests across 30 files, all backend.

7.1 Backend tests

```
npm run test -w backend
```

Mocked Prisma + mocked JWT, no database or network required. Typical run is well under one second.

For per-test verbose output:

```
npm run test -w backend -- --run --reporter=verbose
```

Watch mode while developing:

```
npm run test:watch -w backend
```

7.2 Frontend tests

```
npm run test -w frontend
```

(No frontend tests are written for this release; the script is wired so future tests can be added without changing workspaces.)

7.3 Both workspaces

```
npm run test
```

Runs backend then frontend.

See [test-plan.md](#) for the planned test coverage and [test-results.md](#) for the most recent run evidence.

8. Building for Production

8.1 Build everything

```
npm run build
```

This runs:

- `tsc` in `backend/` → emits `backend/dist/` (CommonJS, compiled from TypeScript).
- `tsc -b && vite build` in `frontend/` → emits `frontend/dist/` (static assets to be served by any web server or CDN).

8.2 Run the production backend

```
npm run start -w backend
```

This requires a valid `.env` at the repository root and a reachable database. The compiled entry point is `backend/dist/app.js`.

8.3 Serve the production frontend

The `frontend/dist/` folder is a static bundle. Locally:

```
npm run preview -w frontend
```

In production, serve `frontend/dist/` from any static host (Render static site, Netlify, an Nginx container, etc.) and point it at the backend's API URL.

9. Deployment Overview

The target deployment is **Render** (free tier). The pattern is:

Component	Render service type	Notes
Backend	Web Service (Docker or Node)	Dockerfile builds and runs <code>npm run start -w backend</code> . Needs <code>DATABASE_URL</code> , <code>JWT_SECRET</code> , <code>FRONTEND_URL</code> set as environment variables in the Render dashboard.
Frontend	Static Site	Build command <code>npm install && npm run build -w frontend</code> , publish directory <code>frontend/dist</code> .
Database	Managed PostgreSQL	The Render-issued connection string goes into the backend's <code>DATABASE_URL</code> env var. Run <code>npm run prisma:migrate -w backend</code> once locally against that URL, or wire it into the deploy command.

CORS: set `FRONTEND_URL` on the backend service to the deployed frontend URL (e.g. `https://propertypilot.onrender.com`) so the browser is allowed to call the API.

Full deployment steps and the cloud-provider justification are tracked separately in `docs/deployment.md` and `docs/cloud-provider-justification.md` (added under Task 4).

10. Troubleshooting

Backend will not start: "JWT_SECRET is required but was not set"

`.env` is missing or `JWT_SECRET` is blank. Copy from `.env.example` and fill it in.

Backend will not start: "JWT_SECRET must be set and at least 16 characters"

Your secret is too short. Replace it with at least 16 characters — preferably 32+ random bytes. See [section 4](#) for a one-liner generator.

Prisma error P1001: "Can't reach database server"

Postgres is not running, or the connection string is wrong.

```
docker compose ps          # Is the container up?
docker compose up -d       # Start it if it isn't
docker compose logs postgres # Look for errors
```

If Docker itself is not running, open Docker Desktop and wait for the system tray icon to stop animating.

Frontend loads but every API call returns 401

You are not signed in, or the JWT in `localStorage` is expired. Open the **Sign in** page and log in again. JWTs are valid for 24 hours.

Frontend loads but every API call fails with a CORS error

`FRONTEND_URL` on the backend does not match the URL the browser is hitting. Update `.env` (locally) or the Render service env var (in production) and restart the backend.

`npm install` fails on Windows with "operation not permitted"

Close VS Code (or any editor) that has the project folder open, then retry. Windows file locks on `node_modules/.bin/` symlinks are the usual culprit.

Migrations drift between the schema and the database

```
npm run prisma:migrate -w backend
```

If Prisma reports drift it cannot reconcile, the safe fix on a local dev DB is to reset (see [section 5.4](#)). **Do not run reset against a production database.**

Tests pass locally but show garbled characters in the terminal output

Windows PowerShell defaults to Windows-1252 and mangles the Vitest checkmark `✓`. Either run Vitest without piping (it prints directly to the console correctly in Windows Terminal/VS Code), or set the session encoding to UTF-8 before piping:

```
$OutputEncoding = [System.Text.Encoding]::UTF8
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
chcp 65001 | Out-Null
```

Port 4000 or 5173 is already in use

Find the process and kill it, or change the port. For the backend, set `PORT` in `.env`. For the frontend, change the port in `frontend/vite.config.ts`.


```
Get-NetTCPConnection -LocalPort 4000 | Select-Object -Property OwningProcess  
Stop-Process -Id <PID>
```

11. Routine Maintenance Tasks

Task	Frequency	Command
Apply new migrations after pulling main	After every <code>git pull</code> that touched <code>backend/prisma/</code>	<code>npm run prisma:migrate -w backend</code>
Reinstall dependencies after a <code>package.json</code> change	After every <code>git pull</code> that touched a <code>package.json</code> or <code>package-lock.json</code>	<code>npm install</code>
Run the test suite before committing	Every commit that touches backend code	<code>npm run test -w backend</code>
Rotate <code>JWT_SECRET</code>	When a secret is suspected compromised, or per the org's rotation policy	Generate a new one, set it as the Render env var, redeploy the backend. All existing sessions will be invalidated.
Back up production data	Per your retention policy	Use Render's built-in Postgres backups, or <code>pg_dump</code> against <code>DATABASE_URL</code>

For end-user instructions, see [user-guide.md](#) .